



GITHUB COPILOT

# Copilot CLI Deep Dive

A hands-on walkthrough using a real Node.js project

7 steps · First interaction → CI integration

# Agenda



What is Copilot CLI?



Install & First Launch



Key Capabilities



Custom Instructions



7-Step Demo



MCP & Agents



CI Pipeline



Before & After

# What is Copilot CLI?

A terminal-native AI coding assistant with agentic capabilities



## Agentic

Plans, executes, and iterates autonomously on complex tasks



## Terminal-Native

Works where you work — right in your shell, no IDE needed



## User Control

Never changes files without your explicit approval

# Installing Copilot CLI

Choose your platform:

 **macOS / Linux**

```
brew install copilot-cli
```

 **Windows**

```
winget install GitHub.Copilot
```

 **Cross-platform (npm)**

```
# Requires Node.js 22+  
npm install -g @github/copilot
```

Available with all Copilot plans. Organization admins must enable the Copilot CLI policy.

# First Launch

## Getting started is 3 commands:

```
# 1. Start an interactive session
copilot

# 2. Authenticate (first time only)
/login

# 3. Ask your first question
Give me an overview of this project.
```

Or run non-interactively with the `-p` flag:

```
copilot -p "How do I fix the failing tests?"
```

## Core Shortcuts

| Shortcut | Action                    |
|----------|---------------------------|
| Esc      | Cancel current operation  |
| Ctrl+C   | Cancel / clear / exit     |
| Ctrl+L   | Clear screen              |
| @        | Mention files for context |
| /        | Show slash commands       |
| ?        | Tabbed help               |
| ↑ ↓      | Command history           |

# How Actions Work

Copilot CLI operates on a trust-and-approve model

## File Trust

When you start a session, Copilot asks about the working directory:

- **Yes, proceed** — trust for this session only
- **Yes, and remember** — trust permanently
- **No, exit** — end the session

Add extra directories mid-session: `/add-dir`

`/path/to/other/project`

Switch working directory: `/cd /path/to/directory`

## Tool Approvals

When Copilot wants to run a command that modifies or executes files:

- **Yes** — allow once, ask again next time
- **Yes, approve for session** — allow this tool for the rest of the session
- **No (Esc)** — reject and give alternate instructions

💡 Rejecting doesn't stop the task — you can give Copilot inline feedback to try a different approach.

# Key Capabilities

## Slash Commands

| Command               | What it does                |
|-----------------------|-----------------------------|
| <code>/help</code>    | Show all commands           |
| <code>/agent</code>   | Select a custom agent       |
| <code>/review</code>  | AI-powered code review      |
| <code>/compact</code> | Compress context history    |
| <code>/usage</code>   | Session stats & token usage |
| <code>/context</code> | Visual token usage overview |
| <code>/resume</code>  | Resume a previous session   |

## Built-in Agents

| Agent                  | Purpose                  |
|------------------------|--------------------------|
| <b>Explore</b>         | Quick codebase Q&A       |
| <b>Task</b>            | Run tests, builds, lints |
| <b>General-purpose</b> | Complex multi-step work  |
| <b>Code-review</b>     | High-signal review       |

## Power Moves

```
# Run shell commands directly  
!git status
```

```
# Resume last session  
copilot --continue
```

```
# Full autonomy mode
```

# Custom Instructions & Agents

## Custom Instructions

Copilot automatically reads project-level instructions:

```
.github/
├── copilot-instructions.md    # repo-wide
├── instructions/
│   └── *.instructions.md     # path-specific
└── agents/
    └── reviewer.agent.md     # custom agents
```

Our demo project already has one:

- Use parameterized queries (never string concat)
- Return `{ errors: [{ field, message }] }`
- Hash passwords with bcrypt
- JWT payload: user id + username

## Agent Scope Levels

| Level          | Location                             |
|----------------|--------------------------------------|
| User           | <code>~/.copilot/agents/</code>      |
| Repo           | <code>.github/agents/</code>         |
| Org/Enterprise | <code>.github-private/agents/</code> |

## Using Agents

```
# Browse available agents
/agent

# Invoke by name in a prompt
Use the security agent to review this PR

# Or via CLI flag
copilot --agent=security --prompt "Audit auth"
```

# Context Management

Keep your sessions efficient as conversations grow

## **/usage**

Session stats: premium requests used, duration, lines edited, token breakdown per model

## **/context**

Visual overview of current token usage — see how much room you have left

## **/compact**

Manually compress conversation history to free up space. Auto-triggers at 95% capacity.

## Session Continuity

`copilot --continue` — resume the last session with full context

`copilot --resume` — browse and select from past sessions

You can even resume Copilot coding agent sessions from GitHub → local CLI

# The 7-Step Demo

1

## First Interaction

Summarize this codebase

2

## Plan Mode

Add user registration

3

## Autopilot

Fix failing tests

4

## Code Review

/review

5

## /delegate

Write real user tests

6

## MCP & Agents

Custom security reviewer

7

## CI Pipeline

Add review to CI



## Result

Complete, secure API

# Step 1: First Interaction

"Summarize this codebase"

# Step 1 — First Interaction

Open the terminal in the project root and ask:

```
# Just ask Copilot to explore the project  
> Summarize this codebase
```

## What Copilot CLI does:

- 📁 Reads `README.md`, `package.json`, and project structure
- 🔍 Scans source files to understand architecture
- 📄 Returns a clear summary of what exists and what's missing

## Why this matters:

- Instant onboarding to any codebase
- Identifies the **tech stack, architecture, and gaps**
- Sets the stage for everything that follows

# What Copilot Finds

## ✅ Working

|                 |                              |
|-----------------|------------------------------|
| Tasks API       | - Full CRUD, 9 tests passing |
| Database        | - SQLite with WAL mode       |
| Health endpoint | - GET /health                |
| Error handling  | - Global error middleware    |

## ⚠️ Incomplete

|                 |                             |
|-----------------|-----------------------------|
| Users API       | - All routes return 501     |
| Auth middleware | - Always returns 401        |
| Validation      | - Passes everything through |
| User tests      | - 11 stubs, no assertions   |

## 🐛 And it finds this...

```
// src/routes/tasks.js - search endpoint
router.get('/search', (req, res) => {
  const tasks = db.prepare(`SELECT * FROM tasks WHERE title LIKE '%${query}%'`).all();
  //                               ⚠️ SQL INJECTION - string interpolation in query
});
```

# Step 2: Plan Mode

"Add user registration with JWT"

## Step 2 — Plan Mode

Ask Copilot to plan a feature before writing code:

```
> Add user registration and login with JWT authentication
```

Copilot creates a plan touching:

| File                   | Changes                                |
|------------------------|----------------------------------------|
| src/models/user.js     | create(), verifyPassword() with bcrypt |
| src/routes/users.js    | Register + Login endpoints             |
| src/middleware/auth.js | JWT token verification                 |
| .env                   | JWT_SECRET configuration               |

The intentional gap it fills:

```
// src/routes/users.js - BEFORE (every route is a stub)
router.post('/register', async (req, res) => {
  // TODO: Implement user registration
```

# Plan Mode — The Result

```
// src/models/user.js — AFTER
const bcrypt = require('bcryptjs');

const User = {
  create({ username, email, password }) {
    const password_hash = bcrypt.hashSync(password, 10);
    return db.prepare(
      'INSERT INTO users (username, email, password_hash) VALUES (?, ?, ?)'
    ).run(username, email, password_hash);
  },

  verifyPassword(username, password) {
    const user = this.getByUsername(username);
    if (!user) return null;
    return bcrypt.compareSync(password, user.password_hash) ? user : null;
  }
};
```

## Key observations:

-  **Parameterized queries** — follows project conventions from `copilot-instructions.md`
-  **bcrypt hashing** — uses the dependency already in `package.json`
-  **Matches existing patterns** — same style as `getByUsername()`

# Step 3: Autopilot

"Fix the failing tests"

# Step 3 — Autopilot

The validation middleware is a no-op — 6 tests are failing:

```
// src/middleware/validate.js - BEFORE
function validateTask(req, res, next) {
  // TODO: Implement validation
  next(); // passes everything through
}
```

> Fix the failing tests

## What Copilot does automatically:

1.  Runs `npm test` — sees 6 failures in `validate.test.js`
2.  Reads the test expectations to understand the required behavior
3.  Implements `validateTask()` with proper validation rules
4.  Re-runs tests — confirms all pass

# The Validation Fix

```
// src/middleware/validate.js - AFTER
function validateTask(req, res, next) {
  const errors = [];
  const { title, description, priority, status } = req.body;

  if (!title || typeof title !== 'string' || !title.trim())
    errors.push({ field: 'title', message: 'Title is required' });
  else if (title.length > 200)
    errors.push({ field: 'title', message: 'Title must be under 200 characters' });

  if (description && description.length > 2000)
    errors.push({ field: 'description', message: 'Description must be under 2000 characters' });

  if (priority && ![ 'low', 'medium', 'high' ].includes(priority))
    errors.push({ field: 'priority', message: 'Invalid priority' });

  if (status && ![ 'pending', 'in_progress', 'done' ].includes(status))
    errors.push({ field: 'status', message: 'Invalid status' });

  if (errors.length) return res.status(400).json({ errors });
  next();
}
```



# Step 4: Code Review

```
/review
```

# Step 4 — Code Review

Use the built-in review command:

```
> /review
```

## Copilot flags the SQL injection:

```
// src/routes/tasks.js - line 23
router.get('/search', (req, res) => {
  const tasks = db.prepare(`SELECT * FROM tasks WHERE title LIKE '%${query}%'`).all();
  //                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  //                                     ▲ CRITICAL: Direct string interpolation - SQL injection vulnerability
});
```

## Suggested fix:

```
// Use parameterized query instead
const tasks = db.prepare('SELECT * FROM tasks WHERE title LIKE ?').all(`%${query}%`);
```

The value is passed as a **parameter**, not interpolated into the SQL string.

# Step 5: /delegate

"Write real tests for users"

# Step 5 — /delegate

The user test file has 11 stubs that test nothing:

```
// tests/users.test.js - BEFORE
test('should hash password before storing', () => {
  expect(true).toBe(true); // placeholder
});
```

```
> /delegate Write real tests for the user registration and login endpoints
```

## Copilot generates tests that:

-  Verify password hashing    Check duplicate prevention    Confirm JWT on login
-  Reject unauthenticated requests    Ensure `password_hash` never exposed

## Why `/delegate`?

- Runs in the **background** — you can keep working
- Generates a full test suite from route signatures + model
- Follows patterns from existing `tasks.test.js`

# Step 6: MCP & Custom Agents

Security Reviewer Agent

# Step 6 — MCP & Custom Agents

## Custom Security Reviewer Agent

```
⌄!— .github/agents/reviewer.agent.md —>
```

```
# Security Reviewer
```

You are a security-focused code reviewer.  
Focus on: SQL injection, XSS, auth bypass,  
secret exposure, and insecure defaults.

Flag any:

- String interpolation in SQL queries
- Missing input sanitization
- Hardcoded secrets or API keys
- Missing auth on sensitive routes
- Overly permissive CORS settings

## What the agent catches:

| Finding                               | File               | Severity                                                                                       |
|---------------------------------------|--------------------|------------------------------------------------------------------------------------------------|
| SQL injection in search               | routes/tasks.js:23 |  Critical |
| JWT SECRET=change-me-to-a-real-secret | .env.example       |  Warning  |

# Step 7: CI Pipeline

Add Code Review to CI

# Step 7 — CI Pipeline

The existing CI workflow has a placeholder:

```
# .github/workflows/ci.yml
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install
      - run: npm test
      # TODO: Add code review step
```

## Ask Copilot to complete it:

> Add a Copilot code review step to the CI pipeline

```
- run: npm test

# Copilot-powered code review on PRs
- name: Copilot Code Review
  if: github.event_name == 'pull_request'
  uses: github/copilot-code-review-action@v1
  with:
    model: gpt-4o
    custom-instructions: |
```

# Before & After

## ● Before Demo

- Users API → all 501
- Auth middleware → always 401
- Validation → no-op
- Tests → 9 pass, 6 fail, 11 stubs
- SQL injection in search
- No CI review

## ● After Demo

- ✓ Users API → register, login, profile
- ✓ Auth middleware → JWT verification
- ✓ Validation → complete with errors
- ✓ Tests → all passing, stubs replaced
- ✓ SQL injection → parameterized query
- ✓ CI → automated Copilot review

# Recap

## 1. Explore

Understand any codebase  
instantly

## 2. Plan

Design before you build

## 3. Autopilot

Fix issues end-to-end

## 4. Review

Catch bugs & vulnerabilities

## 5. Delegate

Background bulk work

## 6. Agents

Custom domain expertise

## 7. CI

Automate in your pipeline

## Key

Real project, real results

The best way to learn Copilot CLI is to use it on **your own codebase**.

# CLI Reference

Your cheat sheet for Copilot CLI

# Slash Commands

## Session & Navigation

| Command                                 | Purpose                          |
|-----------------------------------------|----------------------------------|
| <code>/help</code>                      | Show all available commands      |
| <code>/clear</code> , <code>/new</code> | Clear conversation history       |
| <code>/compact</code>                   | Compress history to free context |
| <code>/context</code>                   | Visual token usage overview      |
| <code>/usage</code>                     | Session stats & metrics          |
| <code>/resume</code>                    | Resume a previous session        |
| <code>/rename</code> NAME               | Rename current session           |
| <code>/session</code>                   | Session info & workspace summary |

## Agents & Tools

| Command                | Purpose                       |
|------------------------|-------------------------------|
| <code>/agent</code>    | Browse & select custom agents |
| <code>/review</code>   | AI-powered code review        |
| <code>/delegate</code> | Delegate to remote PR agent   |
| <code>/plan</code>     | Create implementation plan    |
| <code>/fleet</code>    | Parallel sub-agent execution  |
| <code>/diff</code>     | Review changes in current dir |
| <code>/mcp</code>      | Manage MCP servers            |
| <code>/lsp</code>      | Manage language servers       |

# Keyboard Shortcuts

## Global

| Shortcut               | Action                             |
|------------------------|------------------------------------|
| <code>Esc</code>       | Cancel current operation           |
| <code>Ctrl+C</code>    | Cancel / clear / exit              |
| <code>Ctrl+D</code>    | Shutdown                           |
| <code>Ctrl+L</code>    | Clear screen                       |
| <code>Shift+Tab</code> | Cycle: standard → plan → autopilot |
| <code>@</code>         | Include file in context            |
| <code>!</code>         | Run shell command directly         |

## Editing

| Shortcut                                     | Action                  |
|----------------------------------------------|-------------------------|
| <code>Ctrl+A</code>                          | Beginning of line       |
| <code>Ctrl+E</code>                          | End of line             |
| <code>Ctrl+F</code> /<br><code>Ctrl+B</code> | Forward / back one char |
| <code>Meta+←</code> /<br><code>Meta+→</code> | Move by word            |
| <code>Ctrl+K</code>                          | Delete to end of line   |
| <code>Ctrl+U</code>                          | Delete to start of line |

# Command-Line Flags

## Essentials

```
copilot           # Interactive session
copilot -p "PROMPT" # One-shot prompt
copilot -sp "PROMPT" # Silent one-shot (scripting)
copilot --continue # Resume last session
copilot --resume    # Pick a session to resume
copilot --model=MODEL # Choose AI model
copilot --agent=NAME # Use specific agent
```

## Permissions

```
copilot --allow-all # Allow everything
copilot --yolo        # Same as --allow-all
copilot --allow-tool='shell(git:*)'
copilot --deny-tool='shell(git push)'
copilot --allow-url='github.com'
```

## Configuration

```
copilot --config-dir=PATH # Custom config dir
copilot --log-level=debug  # Set log verbosity
copilot --no-custom-instructions
copilot --no-auto-update
copilot --experimental    # Enable experiments
copilot --screen-reader   # Accessibility mode
copilot --plain-diff      # Disable rich diffs
copilot --streamer-mode   # Hide model/quota info
```

## MCP & Integrations

```
# Add MCP server for this session only
copilot --additional-mcp-config='@mcp.json'

# Disable built-in GitHub MCP
copilot --disable-builtin-mcps

# Enable all GitHub MCP tools
copilot --enable-all-github-mcp-tools
```

# Command-Line Flags (cont.)

## Tool Permission Patterns

| Pattern                          | Matches                   |
|----------------------------------|---------------------------|
| <code>shell(git:*)</code>        | All git commands          |
| <code>write(src/*.ts)</code>     | Write to TS files in src/ |
| <code>read(.env)</code>          | Read .env file            |
| <code>MyMCP(create_issue)</code> | Specific MCP tool         |
| <code>url(github.com)</code>     | URL access                |

```
# Allow all git commands except git push
copilot --allow-tool='shell(git:*)' \
  --deny-tool='shell(git push)'
```

Deny rules always take precedence over allow rules, even with `--allow-all`.

## Scripting & Automation

```
# Autopilot with a limit
copilot -p "Fix tests" \
  --autopilot \
  --max-autopilot-continues=10
```

```
# Share session output
copilot -p "PROMPT" --share=output.md
copilot -p "PROMPT" --share-gist
```

## Non-Interactive Usage

```
# One-shot with silent output (for scripts)
copilot -sp "Explain this error"
```

```
# Pipe into other tools
copilot -sp "Generate a .gitignore for Node" \
  > .gitignore
```

# Environment Variables & Config

## Environment Variables

| Variable                          | Purpose                                         |
|-----------------------------------|-------------------------------------------------|
| <code>COPILOT_MODEL</code>        | Set AI model                                    |
| <code>COPILOT_ALLOW_ALL</code>    | <code>true</code> = allow all permissions       |
| <code>COPILOT_HOME</code>         | Override config dir ( <code>~/.copilot</code> ) |
| <code>COPILOT_GITHUB_TOKEN</code> | Auth token (highest priority)                   |
| <code>GH_TOKEN</code>             | Auth token (medium priority)                    |
| <code>GITHUB_TOKEN</code>         | Auth token (lowest priority)                    |
| <code>COPILOT_AUTO_UPDATE</code>  | <code>false</code> to disable updates           |
| <code>COPILOT_EDITOR</code>       | Editor for <code>Ctrl+G</code>                  |

## Config Files

| Scope | Location                                         |
|-------|--------------------------------------------------|
| User  | <code>~/.copilot/config.json</code>              |
| Repo  | <code>.github/copilot/settings.json</code>       |
| Local | <code>.github/copilot/settings.local.json</code> |

## Key Config Options

```
{
  "model": "claude-sonnet-4",
  "reasoning_effort": "high",
  "theme": "dark",
  "include_coauthor": true,
  "streamer_mode": false,
  "trusted_folders": [
    "/path/to/project"
  ]
}
```

# Thank You

Try it yourself → `gh copilot`



**Copilot CLI**

`gh copilot`



**Docs**

`docs.github.com`



**Demo Repo**

`copilot-cli-demo`